Detección de fraude financiero con Apache Spark y MLlib: Memoria P1 AAGE

David Murado Gómez[1], Álvaro Sieira Rama[2], and Marcos Ramos Gómez[3]

UDC, A Coruña, España
{david.murado,alvaro.sieira.rama,m.rgomez}@udc.es

Resumen. La detección de fraude en transacciones financieras digitales exige tratar grandes volúmenes de datos con un desbalanceo de clases extremo. En este trabajo se emplea una adaptación del conjunto de datos PaySim, con 6.362.620 transacciones de las cuales solo el 0,129 % son fraudulentas, para comparar tres algoritmos de clasificación binaria de Apache Spark MLlib: Regresión Logística, Random Forest y Gradient-Boosted Trees (GBT). Los datos se dividen en entrenamiento (70 %), validación (15 %) y test (15 %), y los modelos se evalúan mediante AUC-PR, AUC-ROC, precisión, recall y F1, dado que el AUC-PR resulta más informativo ante un fuerte desequilibrio de clases. Tras una búsqueda de hiperparámetros, el modelo seleccionado es GBT, que sobre el conjunto de test alcanza un AUC-PR de 0,7547 y un AUC-ROC de 0,9923. Con un umbral de clasificación de 0,30 obtiene una precisión del 87,44 %, un recall del 69,49 % y una puntuación F1 de 0,7744.

Keywords: Detección de fraude · Apache Spark · MLlib · Clases desbalanceadas · Gradient-Boosted Trees.

1. Introducción

El crecimiento masivo de las transacciones financieras digitales ha hecho indispensable el uso de tecnologías de *Big Data* para construir sistemas robustos de detección de fraudes.

Para llevar a cabo este análisis, utilizamos una adaptación del conjunto de datos **PaySim**, un simulador basado en registros reales de transacciones financieras móviles. El objetivo principal del trabajo es aplicar técnicas de ingeniería de características para auditar los balances, entrenar diversos algoritmos de clasificación binaria y evaluar su rendimiento bajo un enfoque de procesamiento distribuido.

2. Análisis Exploratorio y Preprocesamiento

2.1. Descripción de los Datos

El conjunto de datos utilizado contiene 6.362.620 transacciones y 16 columnas originales. La variable objetivo es `isFraud`, que indica si una operación es fraudulenta. El conjunto incluye información sobre el tipo de transacción, su importe, los saldos de origen y destino, y otras variables relacionadas con las operaciones.

Además de las variables originales, el conjunto contiene algunas variables derivadas relacionadas con los saldos y las características de las transacciones. No todas ellas se utilizan como entradas de los modelos: la selección final se realiza atendiendo a las variables disponibles y a su adecuación para la predicción, evitando incorporar información que pueda introducir fuga de datos.

2.2. Distribución de las Clases

El análisis de la variable objetivo `isFraud`, realizado sobre los más de 6,36 millones de registros mediante Spark, revela un fuerte desbalanceo: solo el 0,129 % de las transacciones (8.213 operaciones) son fraudulentas, frente a más de 6,35 millones de operaciones legítimas.

2.3. Preprocesamiento e Ingeniería de Características

Entre las variables numéricas utilizadas se encuentran el importe de la transacción (`amount`), los saldos de origen y destino anteriores a la operación (`oldbalanceOrg` y `oldbalanceDest`) y la variable temporal `step`. También se incorpora `is_merchant_dest` cuando está disponible. La variable categórica `type` se transforma mediante `StringIndexer` y `OneHotEncoder` para poder utilizarla en los modelos.

Los valores numéricos ausentes se tratan mediante un imputador basado en la mediana (`Imputer`). Posteriormente, `VectorAssembler` reúne las variables procesadas en un único vector de características que sirve como entrada a los algoritmos de clasificación.

Aunque el conjunto contiene variables adicionales, como `errorBalanceOrig`, `errorBalanceDest` y `zero_dest_defect`, estas no forman parte del conjunto de características empleado en el entrenamiento operativo. Del mismo modo, los saldos posteriores a la transacción (`newbalanceOrig` y `newbalanceDest`) se consideran únicamente en un análisis exploratorio comparativo, no en el modelo final.

3. Metodología y Modelado

3.1. Metodología Experimental

El proceso experimental se organiza en varias etapas. En primer lugar, se prepara el conjunto de datos y se seleccionan las variables de entrada para predecir la variable objetivo `isFraud`. A continuación, los datos se dividen en tres subconjuntos: entrenamiento (70 %), validación (15 %) y test (15 %), utilizando una semilla fija para facilitar la reproducibilidad de los experimentos.

El conjunto de entrenamiento se utiliza para ajustar los modelos, mientras que el de validación permite comparar su rendimiento y seleccionar configuraciones e hiperparámetros. Una vez elegido el modelo final, su rendimiento se evalúa sobre el conjunto de test, reservado para estimar su capacidad de generalización sobre datos no utilizados durante el entrenamiento ni la selección.

3.2. Algoritmos Seleccionados

Se han seleccionado tres algoritmos de clasificación binaria disponibles en Spark MLlib: Regresión Logística, Random Forest y Gradient-Boosted Trees (GBT). Esta elección permite comparar distintos enfoques de aprendizaje y estudiar su capacidad para identificar operaciones fraudulentas en un conjunto de datos de gran tamaño.

La Regresión Logística se utiliza como modelo de referencia por su sencillez y su capacidad para estimar la probabilidad de fraude. Random Forest combina múltiples árboles de decisión, lo que permite modelar relaciones no lineales entre las variables. Por su parte, GBT construye secuencialmente un conjunto de árboles, tratando de corregir los errores de las iteraciones anteriores.

Los resultados de validación muestran que la Regresión Logística obtiene un AUC-ROC de 0,9142, pero su AUC-PR es de 0,0182 y sus valores de precisión, recall y F1 son nulos con el umbral de clasificación utilizado. Por tanto, aunque el modelo distingue entre clases mejor que un clasificador aleatorio según el AUC-ROC, presenta una capacidad práctica muy limitada para identificar fraudes con la configuración evaluada.

3.3. Métricas de Evaluación

Para evaluar el rendimiento de los modelos se utilizan el área bajo la curva precisión-recall (AUC-PR), el área bajo la curva ROC (AUC-ROC), la precisión, el recall y la puntuación F1. La elección de estas métricas responde al fuerte desbalanceo de clases presente en el conjunto de datos, donde las transacciones fraudulentas representan una proporción muy pequeña del total.

El AUC-PR resulta especialmente relevante, ya que permite analizar el equilibrio entre precisión y recall para distintos umbrales de clasificación, centrándose en la detección de la clase minoritaria. El AUC-ROC mide la capacidad del modelo para distinguir entre transacciones legítimas y fraudulentas a diferentes umbrales. Sin embargo, puede ofrecer una visión demasiado optimista cuando existe un desbalanceo tan elevado.

La precisión mide qué proporción de las transacciones identificadas como fraudulentas son realmente fraudes, mientras que el recall indica qué proporción de los fraudes reales consigue detectar el modelo. Por su parte, la puntuación F1 combina ambas métricas mediante su media armónica. Estas medidas permiten valorar tanto la fiabilidad de las alertas como la capacidad para detectar operaciones fraudulentas que, de otro modo, pasarían inadvertidas.

3.4. Optimización de Hiperparámetros

Para mejorar el rendimiento de los modelos, se realiza una búsqueda de hiperparámetros sobre Random Forest y Gradient-Boosted Trees (GBT) mediante las herramientas de ajuste de Spark MLlib. Se utiliza `ParamGridBuilder` para definir distintas combinaciones de parámetros y `TrainValidationSplit` para seleccionar la configuración con mejor rendimiento sobre una partición interna de validación.

En Random Forest se exploran diferentes valores para el número de árboles (`numTrees`: 40 y 60) y la profundidad máxima (`maxDepth`: 5 y 7). En GBT se ajustan el número de iteraciones (`maxIter`: 20 y 35) y la profundidad máxima de los árboles (`maxDepth`: 3 y

5). La selección se realiza utilizando el AUC-PR como métrica de evaluación, dada su relevancia en problemas con un fuerte desequilibrio de clases.

Una vez obtenidas las configuraciones seleccionadas, los modelos se comparan sobre el conjunto de validación externo. En esta ejecución, la optimización de GBT no produce una mejora en las métricas respecto al modelo inicial: ambos obtienen un AUC-PR de 0,7317. La configuración optimizada de Random Forest alcanza un AUC-PR de 0,6829, frente a 0,6748 del modelo inicial, aunque su recall es ligeramente inferior.

Finalmente, se selecciona el modelo con mejor AUC-PR en la comparación de validación y se estudian distintos umbrales de clasificación para encontrar un equilibrio entre precisión y recall. La evaluación definitiva se realiza sobre el conjunto de test, reservado para estimar el rendimiento sobre datos no utilizados en la selección del modelo ni del umbral.

4. Resultados y Discusión

4.1. Comparativa de Modelos

Para comparar los modelos se utilizan las métricas calculadas sobre el conjunto de validación. La Tabla 1 recoge los resultados de los modelos iniciales y de las configuraciones optimizadas, cuando corresponde.

Tabla 1. Resultados de los modelos sobre el conjunto de validación.

Modelo	AUC-PR	AUC-ROC	Precisión	Recall	F1
Regresión Logística	0,0182	0,9142	0,0000	0,0000	0,0000
Random Forest	0,6748	0,9694	0,9972	0,2927	0,4526
Random Forest optimizado	0,6829	0,9751	0,9972	0,2862	0,4447
GBT	0,7317	0,9914	0,9264	0,5765	0,7107
GBT optimizado	0,7317	0,9914	0,9264	0,5765	0,7107

GBT obtiene el mayor AUC-PR y el mayor recall entre los modelos comparados, con un equilibrio entre precisión y detección de fraudes reflejado en su puntuación F1. Random Forest presenta una precisión muy elevada, pero un recall considerablemente menor, por lo que deja sin detectar una parte importante de las operaciones fraudulentas. La Regresión Logística ofrece los peores resultados en AUC-PR y en las métricas calculadas con el umbral empleado.

La optimización de Random Forest mejora ligeramente el AUC-PR, aunque no aumenta el recall ni la puntuación F1. En GBT, la configuración optimizada obtiene las mismas métricas que el modelo inicial. Por tanto, en esta ejecución, el modelo GBT inicial es seleccionado por su rendimiento de validación sin que la búsqueda de hiperparámetros aporte una mejora observable en dichas métricas.

4.2. Análisis de Resultados

El análisis se centra en el comportamiento del modelo seleccionado sobre el conjunto de test, que no se ha utilizado durante el entrenamiento ni durante la selección de hiperparámetros. El modelo final, GBT, obtiene un AUC-PR de 0,7547 y un AUC-ROC de 0,9923.

Con el umbral de clasificación seleccionado de 0,30, el modelo alcanza una precisión del 87,44 %, un recall del 69,49 % y una puntuación F1 de 0,7744. Estos resultados muestran que el modelo identifica una parte importante de las operaciones fraudulentas manteniendo una proporción elevada de predicciones positivas correctas.

La matriz de confusión registra 870 fraudes detectados (TP), 382 fraudes no detectados (FN), 125 falsas alarmas (FP) y 954.695 transacciones legítimas correctamente clasificadas (TN).

La elección del umbral implica un compromiso entre precisión y recall: modificarlo puede cambiar tanto el número de fraudes detectados como el de falsas alarmas. El valor 0,30 se selecciona a partir del conjunto de validación, maximizando la puntuación F1, y después se evalúa en test.

También debe considerarse que la partición aleatoria de los datos no garantiza que el rendimiento se mantenga ante cambios temporales en los patrones de fraude. Por ello, sería conveniente realizar evaluaciones adicionales con datos de periodos posteriores antes de utilizar el modelo en un entorno real.

5. Conclusiones y Trabajo Futuro

El objetivo de esta práctica ha sido desarrollar y comparar distintos modelos de clasificación binaria mediante Apache Spark y MLlib para detectar operaciones fraudulentas en un conjunto de datos de gran tamaño. Se han evaluado Regresión Logística, Random Forest y Gradient-Boosted Trees (GBT).

Los resultados obtenidos muestran que GBT es el modelo seleccionado. Sobre el conjunto de test alcanza un AUC-PR de 0,7547 y un AUC-ROC de 0,9923. Con un umbral de clasificación de 0,30, obtiene una precisión del 87,44 %, un recall del 69,49 % y una puntuación F1 de 0,7744. Además, detecta 870 fraudes y genera 125 falsas alarmas, aunque deja sin detectar 382 operaciones fraudulentas.

Como trabajo futuro, se podría estudiar el efecto de otras estrategias para tratar el desequilibrio de clases, ampliar la búsqueda de hiperparámetros y analizar nuevas variables que mejoren la identificación de operaciones fraudulentas. También sería interesante comparar distintos umbrales según los costes asociados a los fraudes no detectados y a las falsas alarmas.

Por último, convendría evaluar el modelo sobre datos de periodos posteriores para comprobar su estabilidad frente a posibles cambios temporales en los patrones de fraude. La partición aleatoria utilizada en esta práctica permite realizar una comparación experimental, pero no garantiza por sí sola que el rendimiento se mantenga en un escenario real. Por tanto, los resultados deben interpretarse en el contexto del conjunto de datos y del procedimiento experimental empleados.